

ProjectOS

Test Report — multi-tenant SaaS with a server-side AI layer

Build under test: [my-major-project-rho.vercel.app](#)

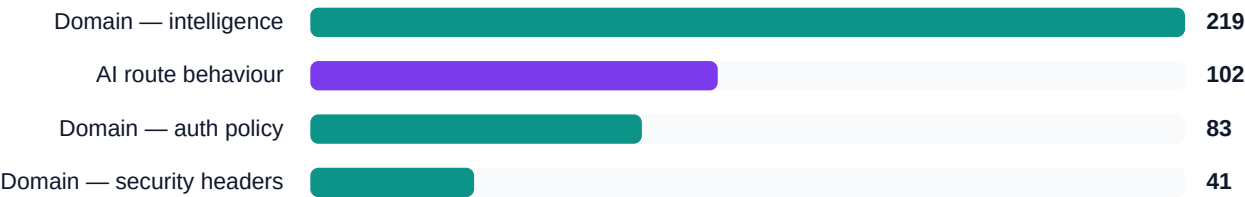
445

assertions passed

0 failed · 0 defects found in application code

Verified from a clean checkout with no database and no prisma generate

Coverage at a glance



Reproduce any of it with `npm test`. The AI suite needs no database and no API key.

1. Method

Four layers were tested, deliberately different in kind: pure domain logic against real inputs; the AI route handlers running as real code; a scripted audit of architecture and tenant isolation; and the live deployment over HTTP.

The AI tests import the **actual route handlers** and call them. Only three modules are substituted, through path mappings in a test-only tsconfig — so **nothing under src/ was modified to make tests pass**. The system prompts are the real ones, re-exported from source.

Substituted	Replaced with	Why
@/lib/db	In-memory store	No database; lets a second tenant exist for isolation probes
@/lib/auth/context	Scriptable session	One test runs as ADMIN, the next as MEMBER
@/lib/ai	Scriptable model	The model can return malformed JSON or fail mid-stream on demand

2. The AI layer under adversarial input

Model output is treated as hostile. Every case below is handled correctly — this is the core of the AI testing.

The model returns...	Result
Well-formed JSON	200 — fields pass through
Wrapped in a ``json fence	Recovered
Wrapped in prose (“Sure! Here is...”)	Recovered

A single-element array	Recovered (deliberate — see §5)
A multi-task array	422 — never guesses which task
No JSON at all	422 “Could not parse task from that text”
No title, or a whitespace title	422
priority: “bogus”	Falls back to MEDIUM
priority: “urgent”	Normalised to URGENT
dueDate: “next friday”	null — never guesses a date
dueDate: “20-08-2026”	null — non-ISO rejected
A 400-character title	Bounded to 180
Wrong-typed fields (priority: 123)	Degrade safely
The call throws	502

3. Data egress — what actually reaches the model

Asserted directly on the outgoing prompt string, not assumed: **no task titles** in the forecast or retrospective context, and **no organisation id, project id or task ids** in any AI call. The browser never contacts Anthropic — CSP connect-src ‘self’ is intact and every model call is server-side.

4. Tenant isolation and security audit

Check	Result
Every non-auth API route uses withTenantGuard	Pass
93 org-owned Prisma queries filter on organizationId	Pass
organizationId never read from a request body or param	Pass
Another tenant’s project returns 404, not 403	Pass
A milestone in a different project of the same tenant returns 404	Pass
src/lib/domain never imports the AI layer or Prisma	Pass
src/lib/ai has no Prisma, React or tenant imports	Pass
No AI route writes to the database	Pass
No live API key in tracked files; no NEXT_PUBLIC secret	Pass
CSP connect-src ‘self’ intact	Pass

Five items were flagged by the audit script and each was investigated individually. **All five were false positives** — for example, the two routes without withTenantGuard authenticate with requireAuthenticatedUser() instead, which is correct: password changes are user-level, and an organisation is created before any tenant exists.

5. Findings and judgement calls

No defects were found in application code. Three decisions are stated explicitly rather than left implicit:

Single-element array is recovered, not rejected	If the model returns <code>[{...}]</code> the inner object is used: the intent is unambiguous and the user confirms before anything is created. A multi-task array is still refused.
Milestone closure time is inferred	The schema has no <code>completedAt</code> column, so closure is derived from the latest completed task. The proper fix is a real column written on status change.
No PATCH route for milestones	Nothing in the running app marks a milestone DONE or MISSED yet, so the status must arrive via seed data.

6. What was not tested, and why

Stated plainly so the coverage claim is honest. **No browser click-through** — the components were verified by type-checking and review, not by rendering. **No live authenticated API calls**. **No real Anthropic responses** — the model is doubled, which is stronger for testing failure paths but cannot judge whether the generated prose reads well; that needs a human with a real key. **No database-level execution** — migrations and cascade rules were verified by schema review.

7. Before presenting

1. Confirm the newest routes (forecast, retrospective) are included in the deployed build.
2. Rotate the database password and the Anthropic API key — both passed through chat during development.
3. Set `secure: true` on the session cookie for production HTTPS.
4. Optionally add the milestone PATCH route so the retrospective can be demonstrated end to end.